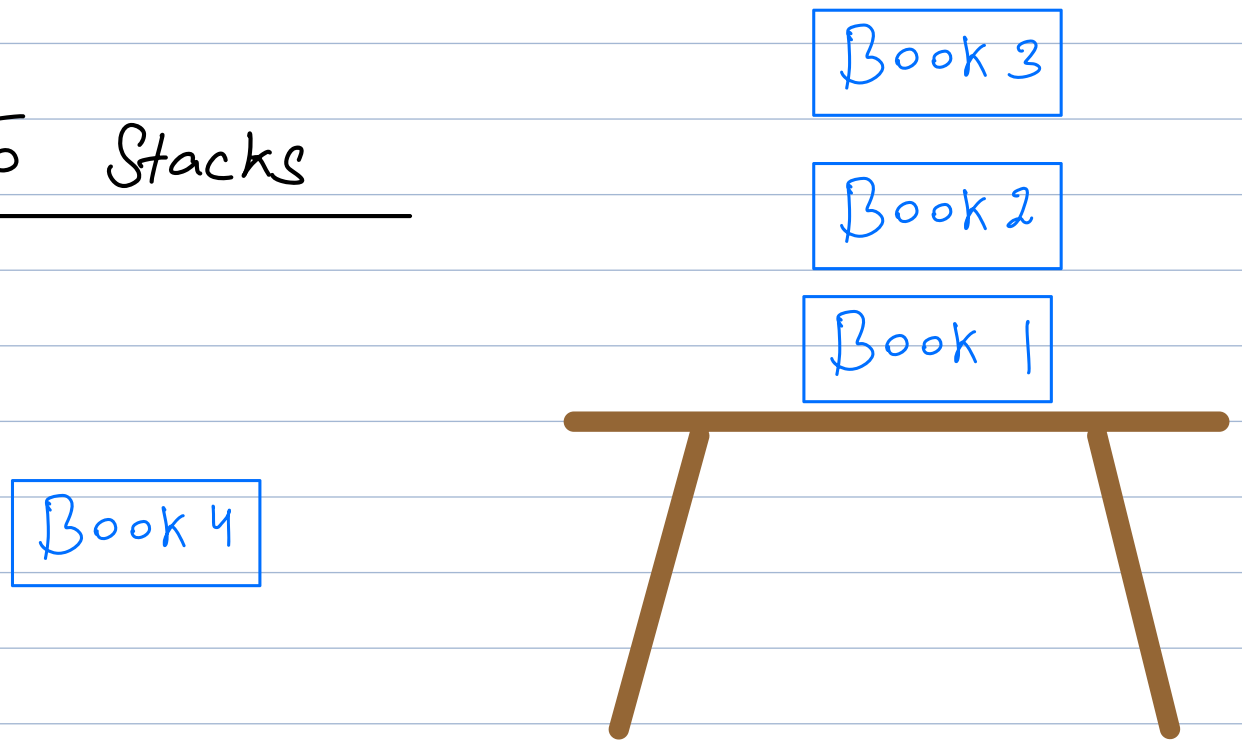
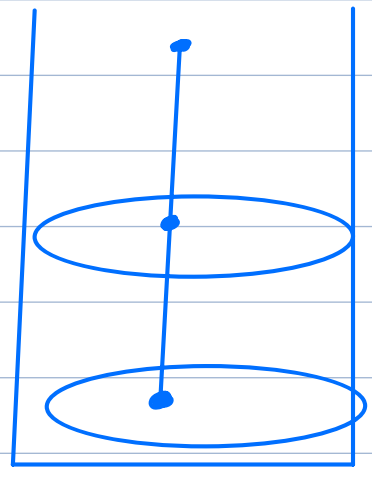
Today's Agenda:-

1. Types of Memory
2. Memory Management.
3. Sorting
4. Questions on Sorting.

Starting 7:05

Introduction To Stacks





An Idli Maker

Properties of Stack:-

1. I have access only to the topmost element that is placed on the stack.

2. Follows the LIFO principle.

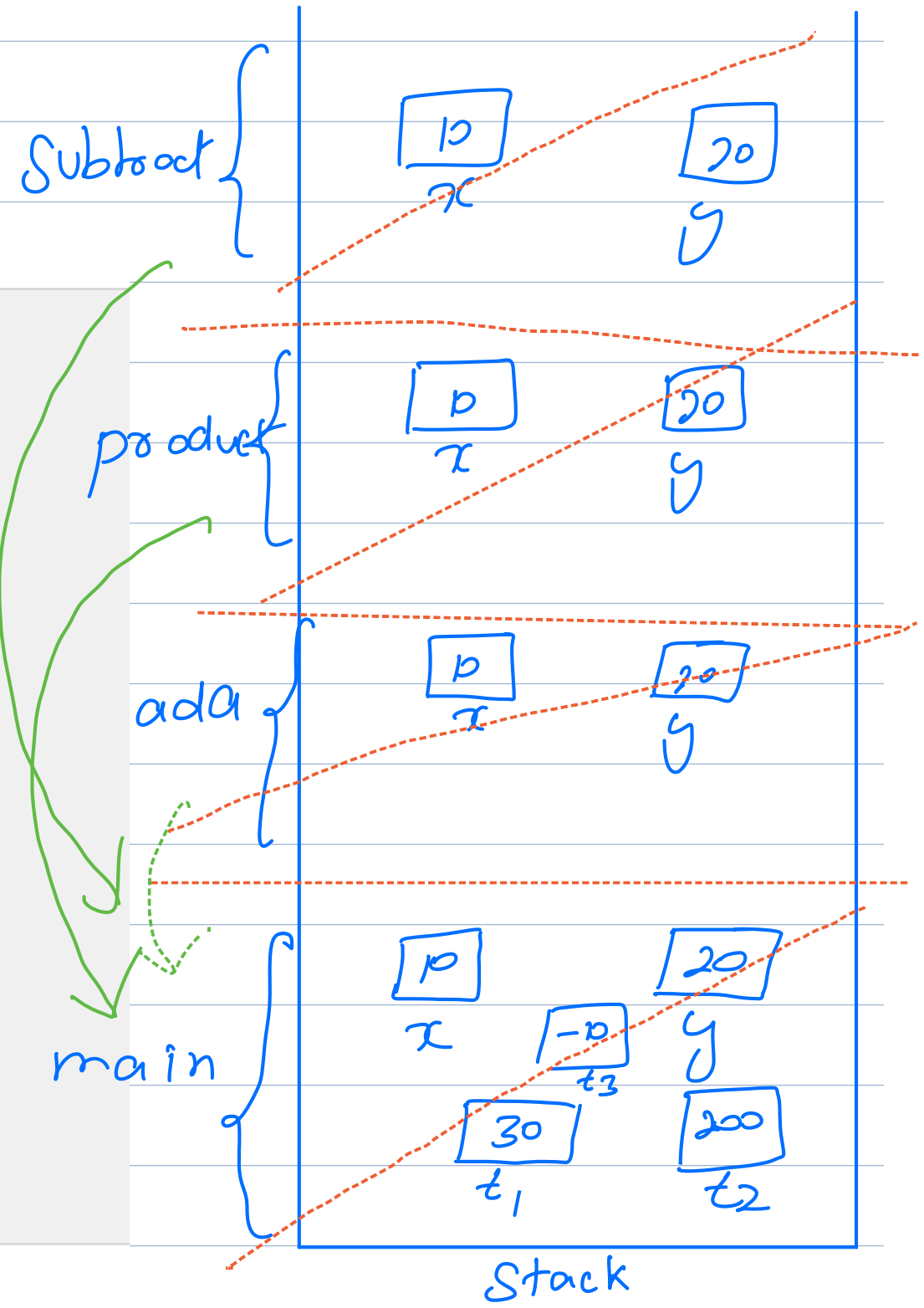
(Last In First Out).

println { o/p:- -10).

Call Stack

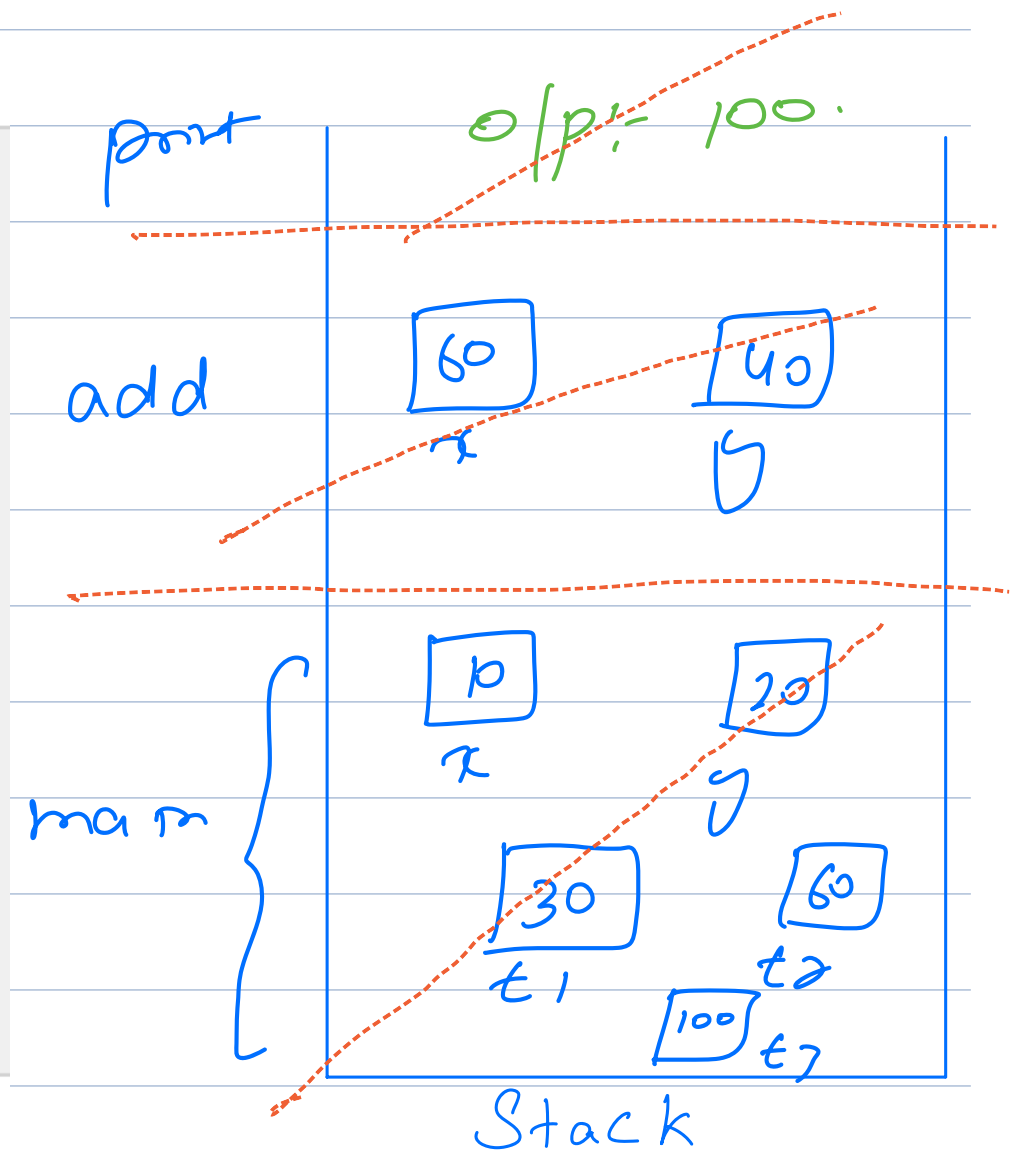
(1)

```
int add(int x, int y) {
    return x + y;
}

int product(int x, int y) {
    return x * y;
}

int subtract(int x, int y) {
    return x - y;
}

public static void main() {
    int x = 10;
    int y = 20;
    int temp1 = add(x, y);
    int temp2 = product(x, y);
    int temp3 = subtract(x, y);
    System.out.println(temp3);
}
```



(2)

```
int add(int x, int y) {
    return x + y;
}

public static void main() {
    int x = 10;
    int y = 20;
    int temp1 = add(x, y);
    int temp2 = add(temp1, 30);
    int temp3 = add(temp2, 40);
    System.out.println(temp3);
}
```



Types of Memory in Java.

Stacks	Heaps
1. The function calls.	Heap Stores the actual containers like arrays, objects, a L.L. node etc;.
2. All the primitive data types int, char, float, double etc.	
3. The reference variable.	

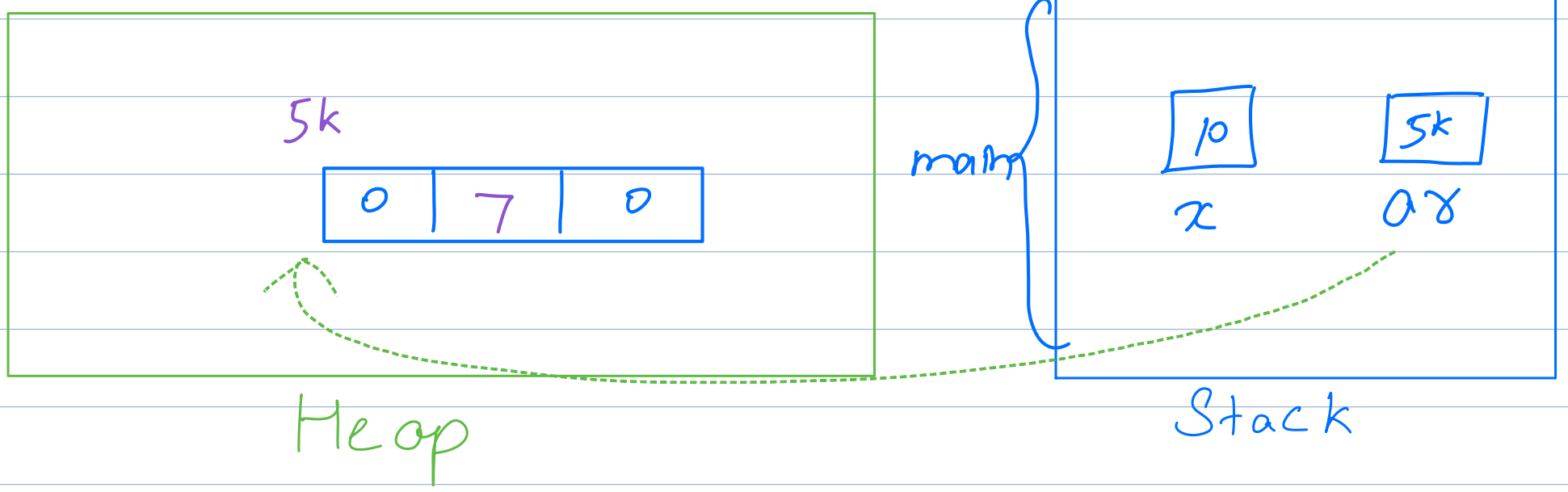
Ex - 1.

o/p

5k.

0

```
public static void main() {
    int x = 10;
    int[] ar = new int[3];
    System.out.println(ar); // #ad1
    System.out.println(ar[2]); // 0
    ar[1] = 7;
}
```



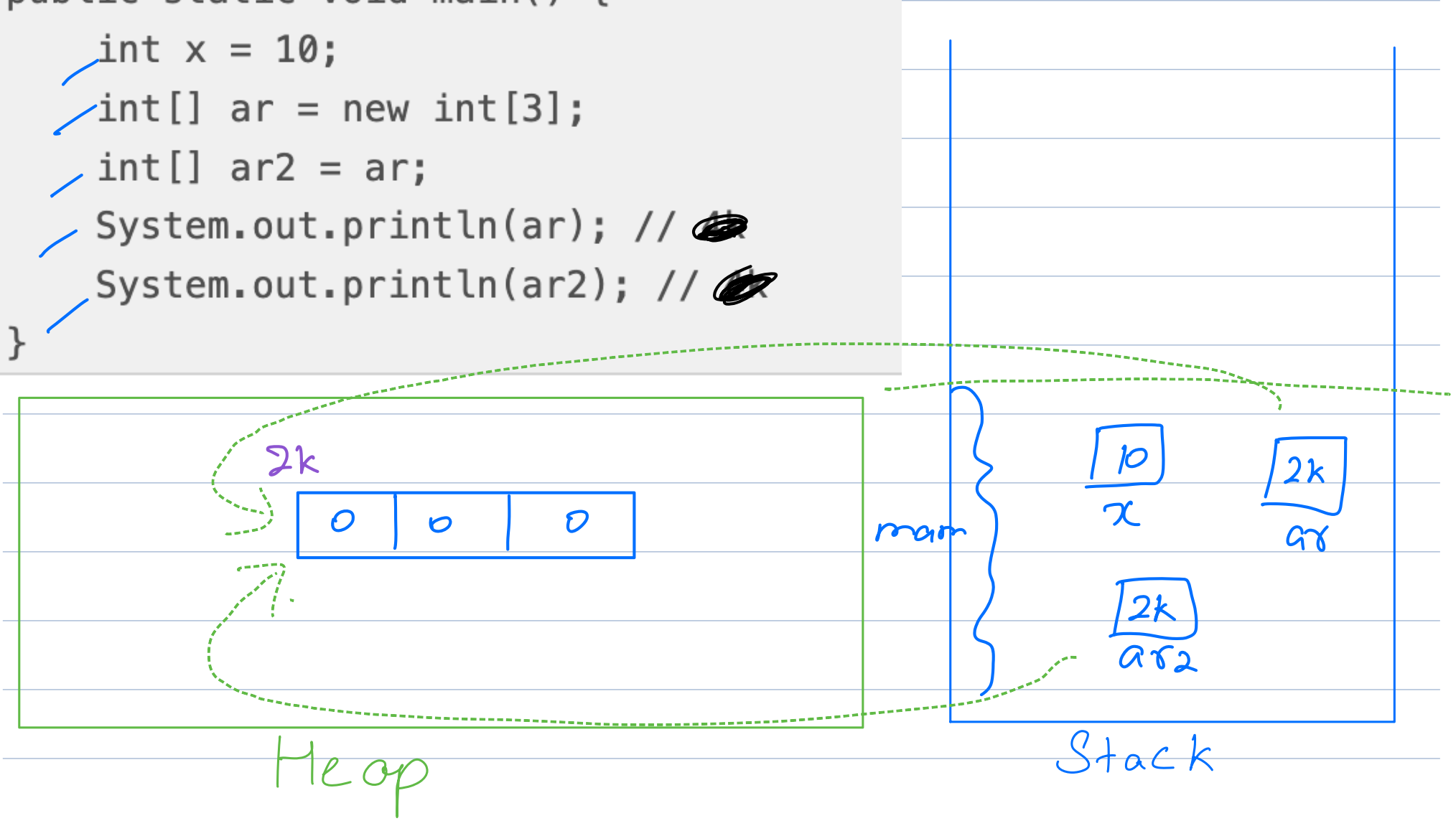
Ex:- 2

o/p

2k

2k.

```
public static void main() {
    int x = 10;
    int[] ar = new int[3];
    int[] ar2 = ar;
    System.out.println(ar); // 4k
    System.out.println(ar2); // 4k
}
```



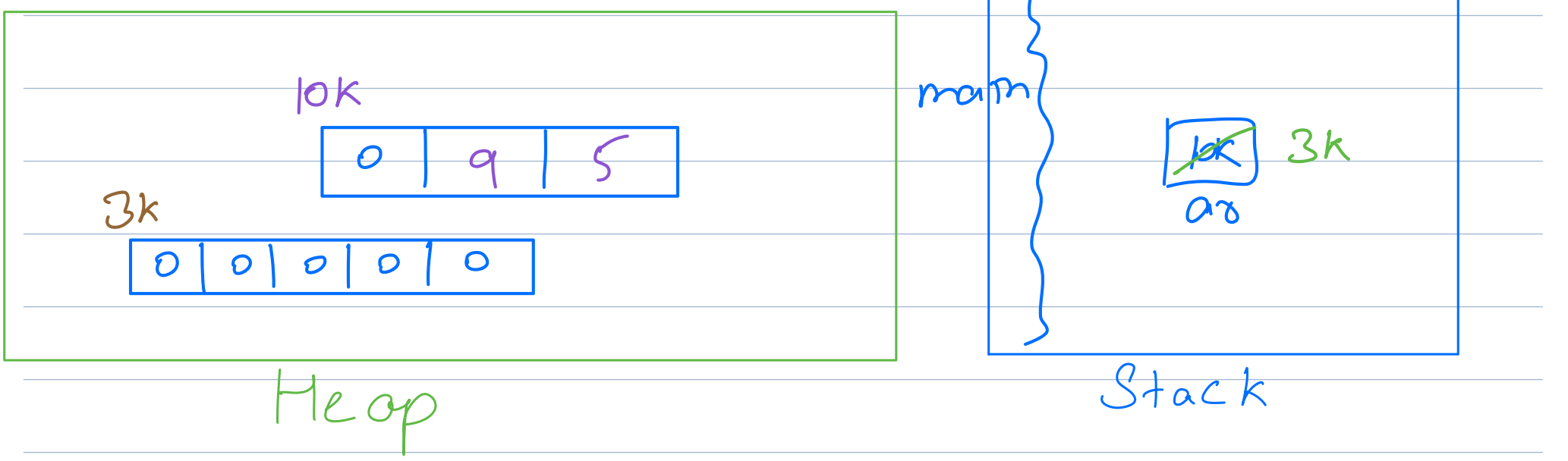
Ex:- 3.

O/P

10k

3k.

```
public static void main() {
    int[] ar = new int[3];
    System.out.println(ar); [redacted]
    ar[1] = 9;
    ar[2] = 5;

    ar = new int[5];
    System.out.println(ar); [redacted]
}
```



Ex: 4

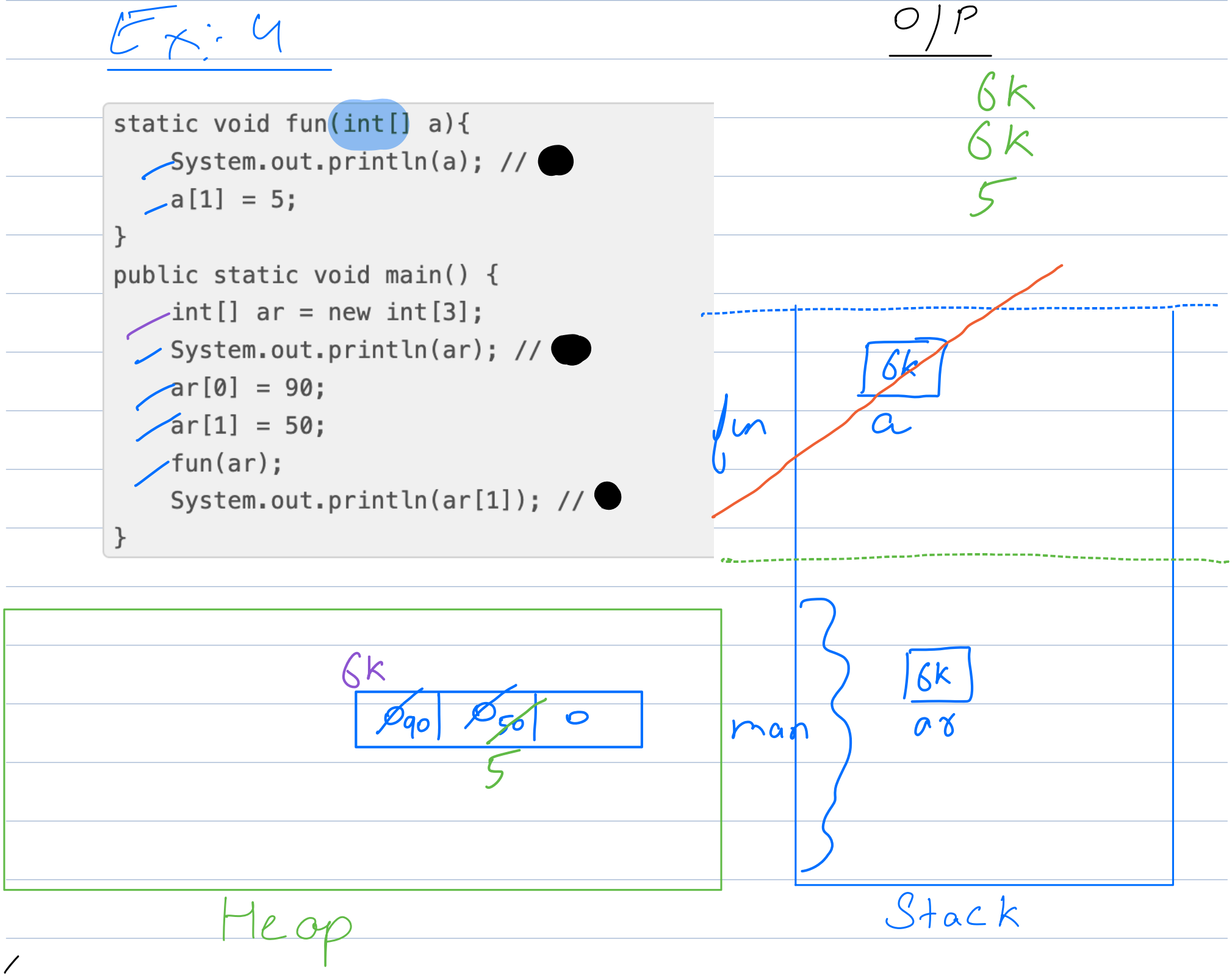
O/P

6k

6k

5

```
static void fun(int[] a){
    System.out.println(a); // [redacted]
    a[1] = 5;
}
public static void main() {
    int[] ar = new int[3];
    System.out.println(ar); // [redacted]
    ar[0] = 90;
    ar[1] = 50;
    fun(ar);
    System.out.println(ar[1]); // [redacted]
}
```

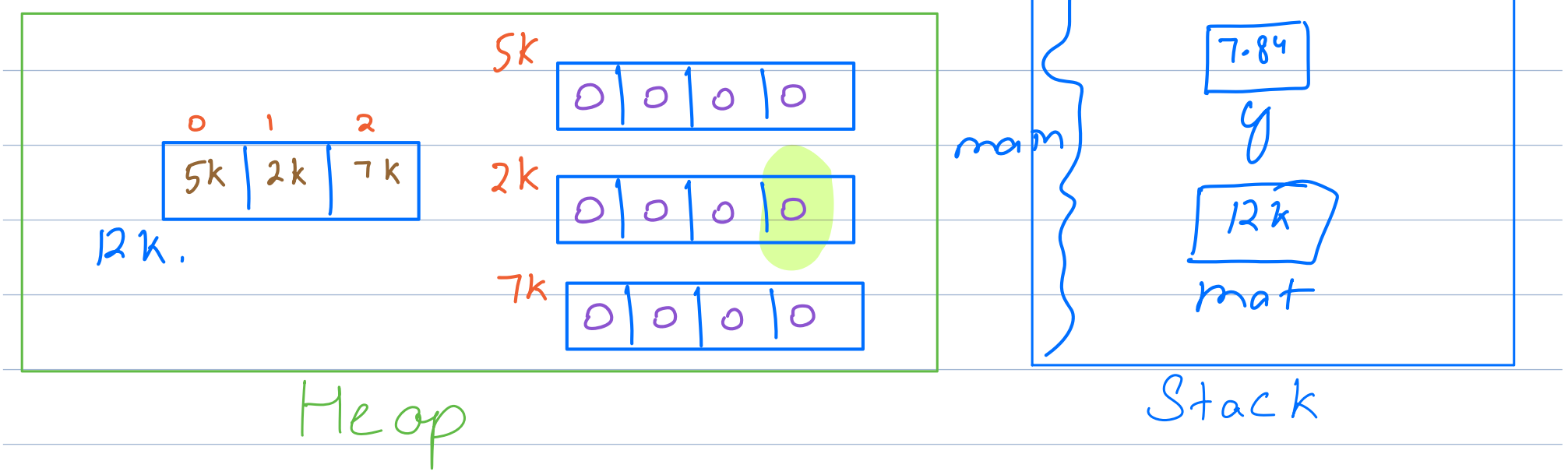


Ex:- 5

o/p

12k
2k.
0.

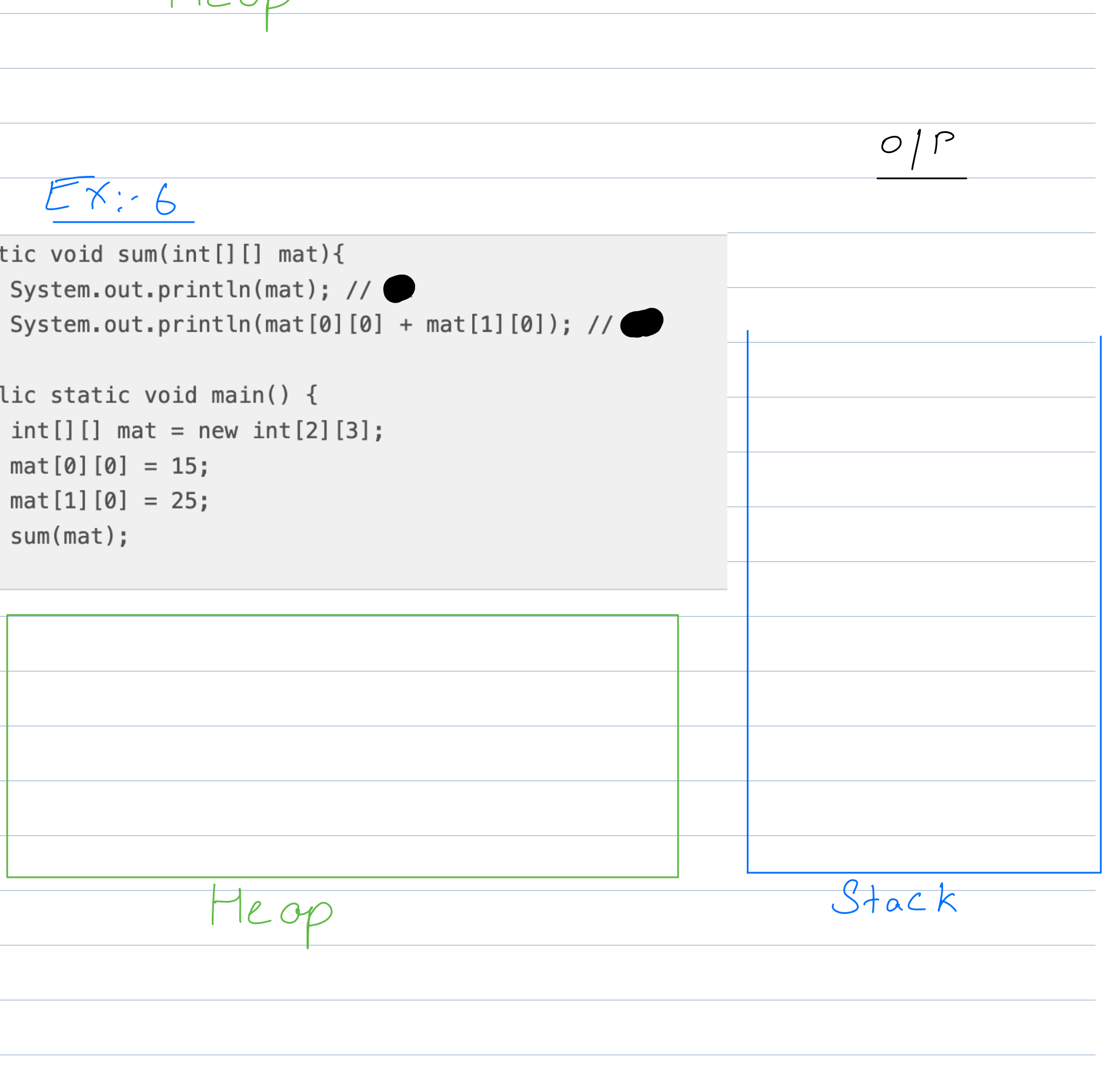
```
public static void main() {
    float y = 7.84f;
    int[][] mat = new int[3][4];
    System.out.println(mat); //
    System.out.println(mat[1]); //
    System.out.println(mat[1][3]); //
}
```



o/p

Ex:- 6

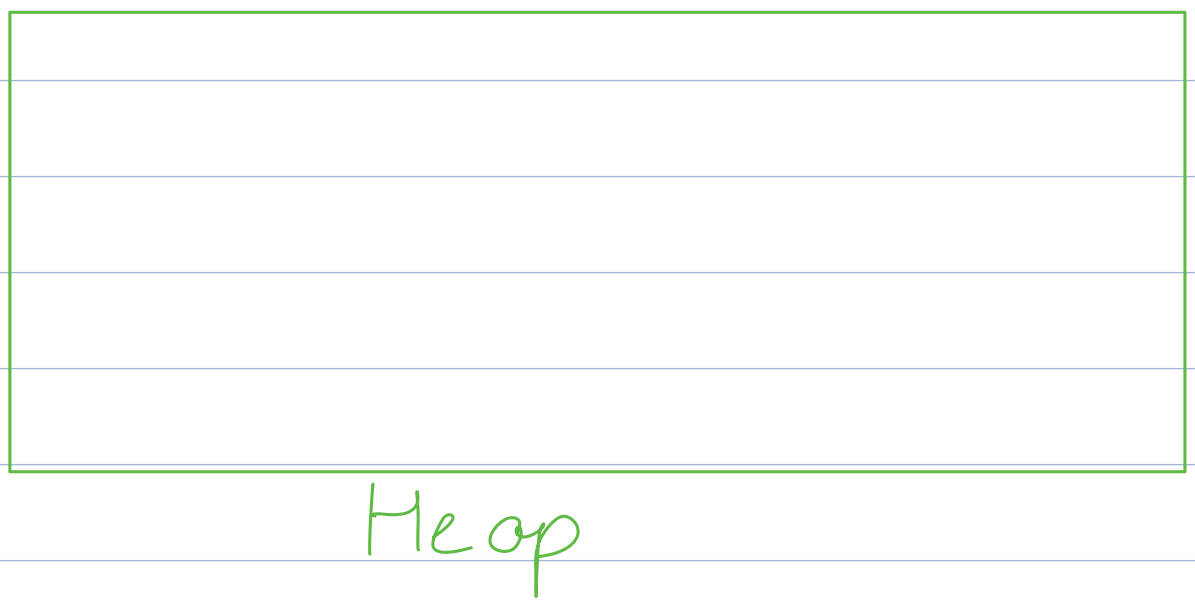
```
static void sum(int[][] mat){
    System.out.println(mat); //
    System.out.println(mat[0][0] + mat[1][0]); //
}
public static void main() {
    int[][] mat = new int[2][3];
    mat[0][0] = 15;
    mat[1][0] = 25;
    sum(mat);
}
```



Quizzes

Q-1.

```
static void change(int a) {
    a = 50;
}

public static void main(String args[]) {
    int a = 10;
    change(a);
    System.out.println(a);
}
```

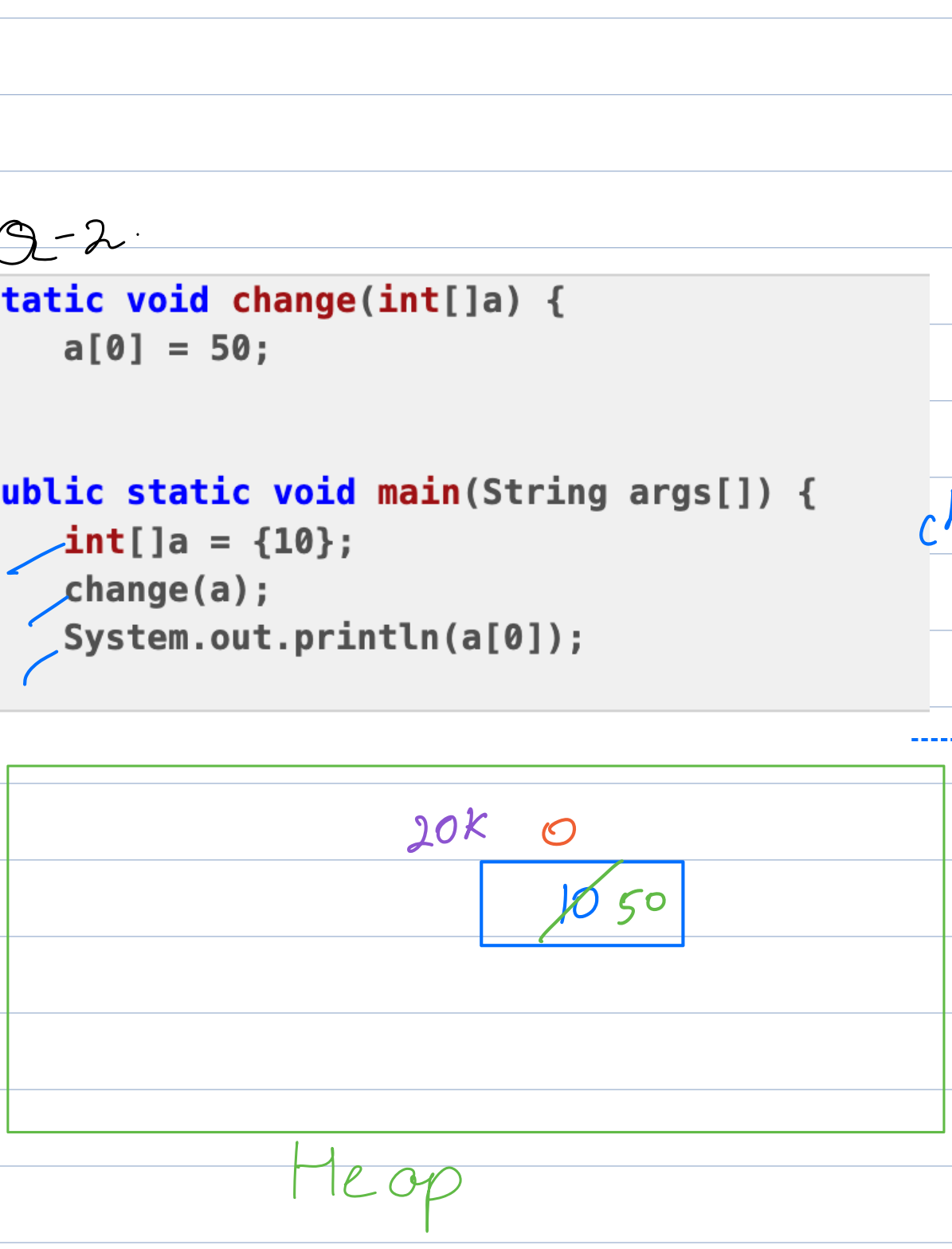


o/p :- 10

o/p :- 50

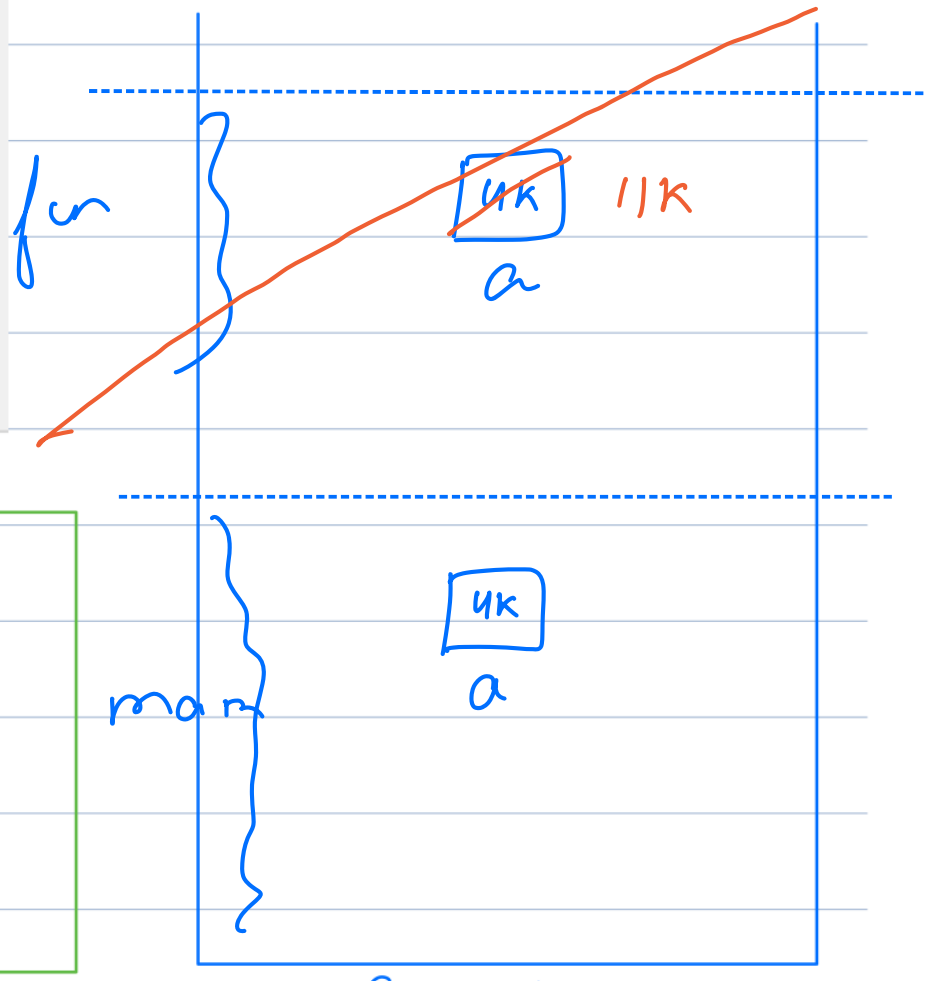
Q-2.

```
static void change(int[]a) {
    a[0] = 50;
}

public static void main(String args[]) {
    int[]a = {10};
    change(a);
    System.out.println(a[0]);
}
```



3.

```
static void fun(int[] a) {
    a = new int[1];
    a[0] = 100;
}
public static void main() {
    int[] a = {10, 20, 30};
    fun(a);
    System.out.println(a[0]);
}
```

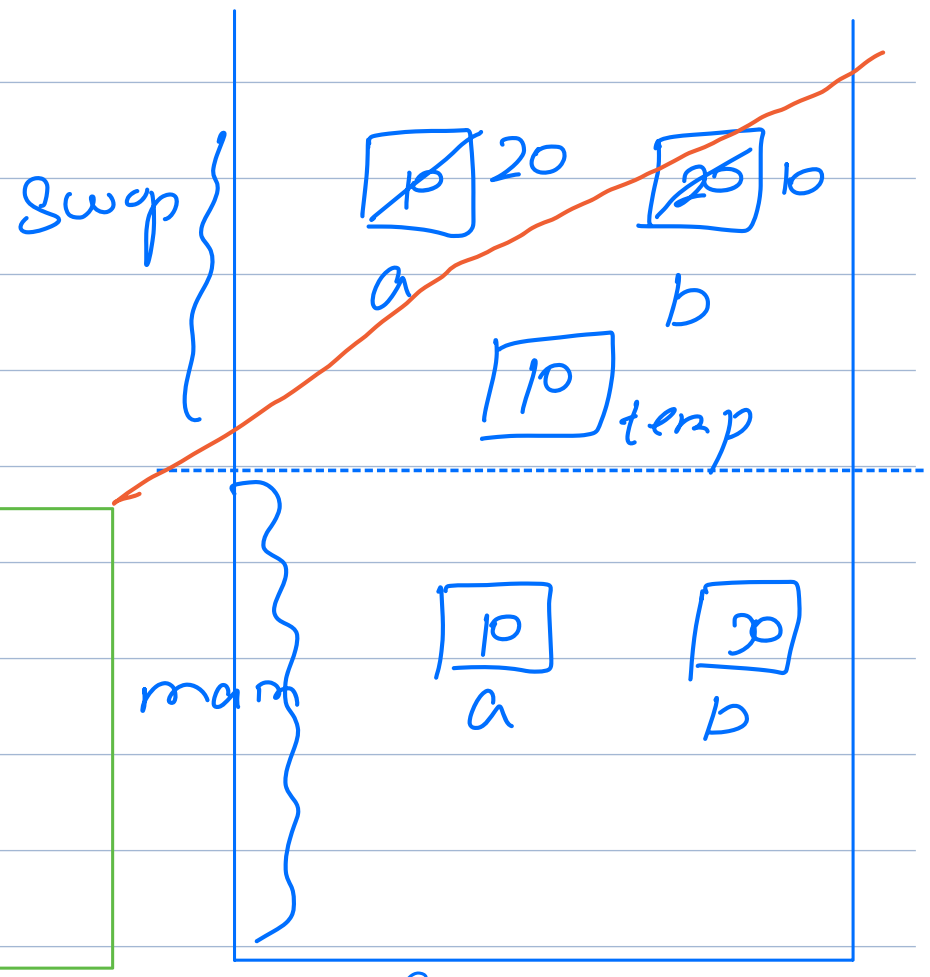


o/p :- 10.

4.

```java
static void swap(int a,int b) {
    int temp = a;
    a = b;
    b = temp;
}

public static void main(String args[]) {
    int a = 10;
    int b = 20;
    swap(a,b);
    System.out.println(a + " " + b);
}
```

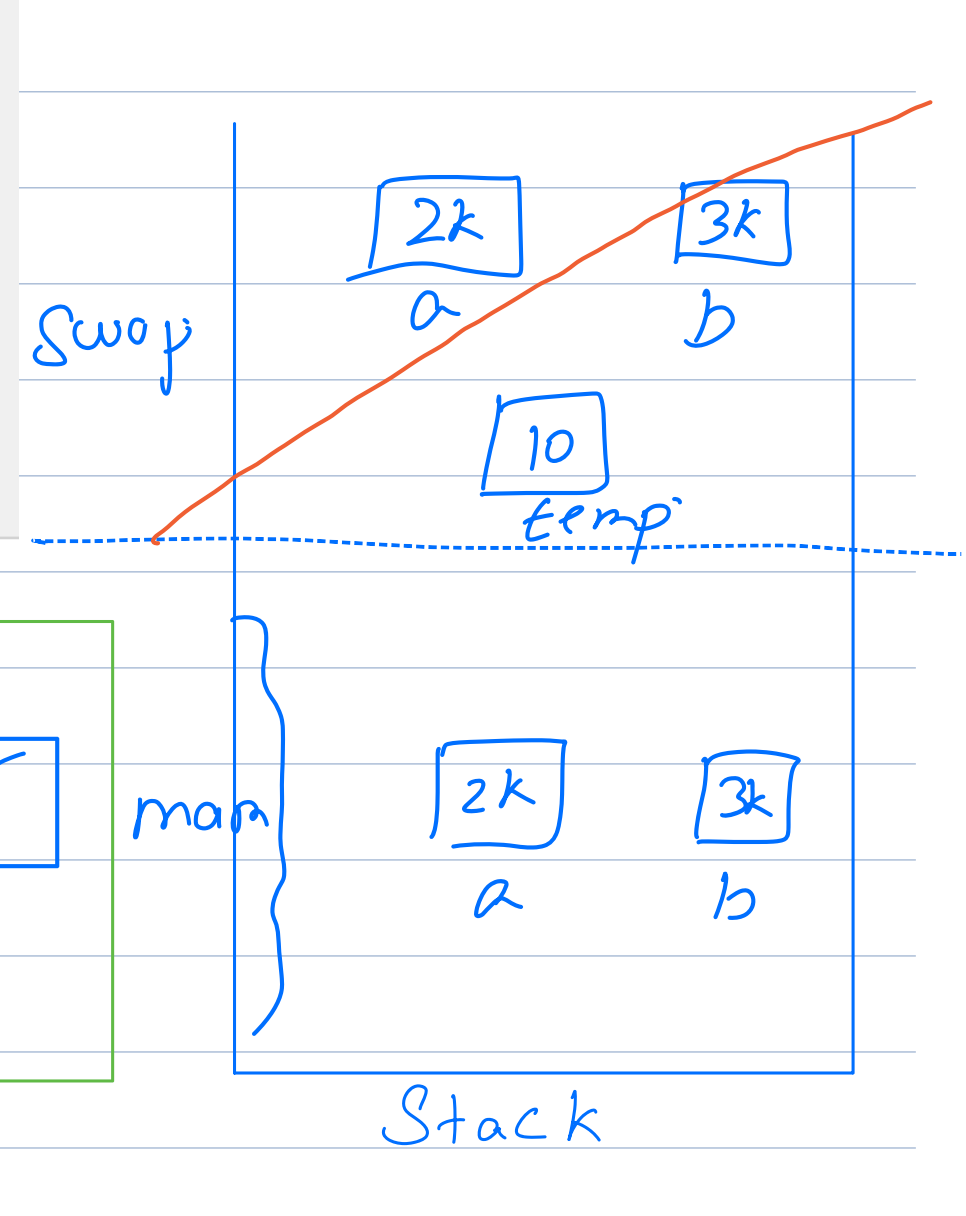
10 20.



Q-5.

o/p :- 20 10.

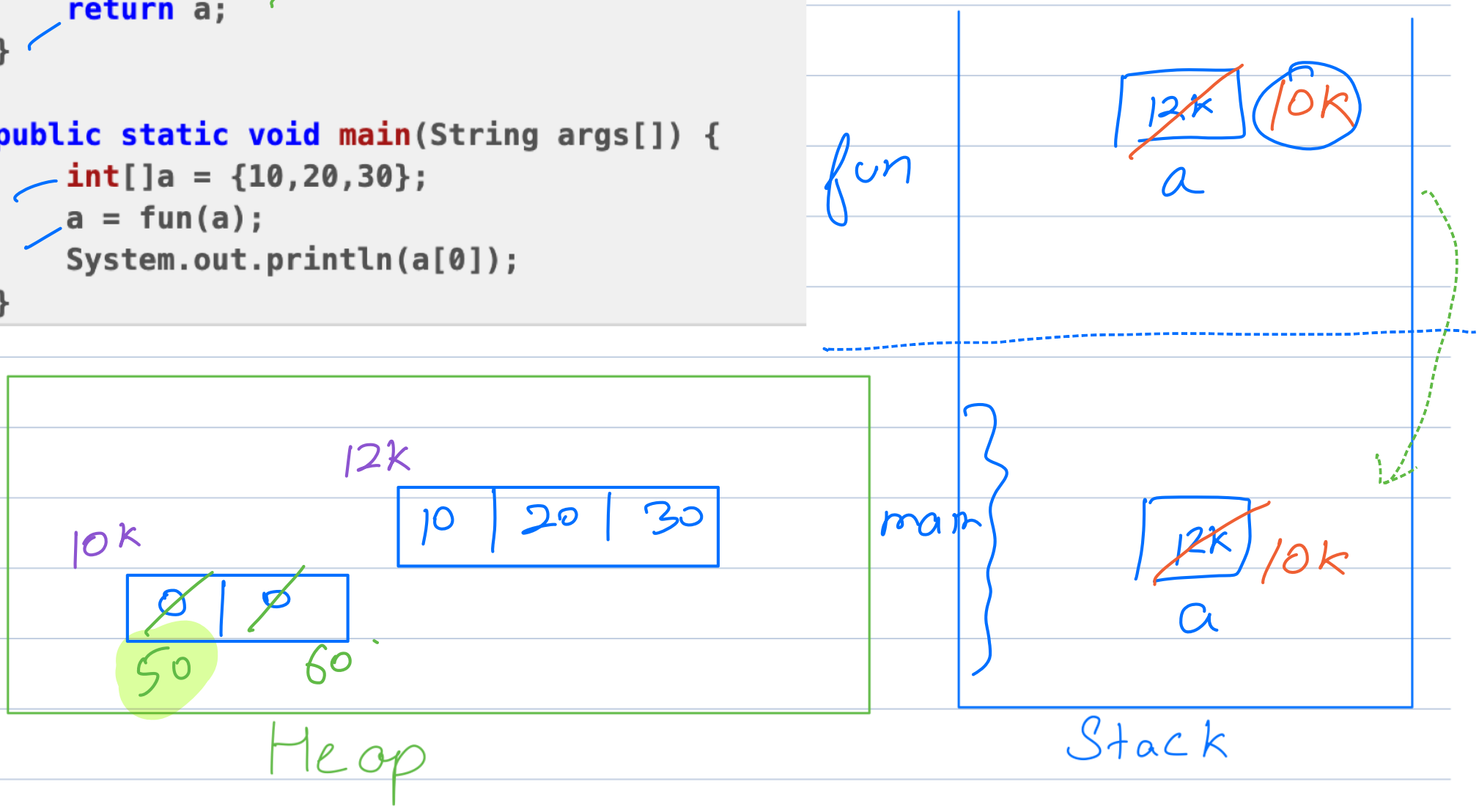
```java
static void swap(int[]a,int[]b) {
    int temp = a[0];
    a[0] = b[0];
    b[0] = temp;
}

public static void main(String args[]) {
    int[]a = {10};
    int[]b = {20};
    swap(a,b);
    System.out.println(a[0] + " " + b[0]);
}
```



6.

```
static int[] fun(int[]a) {
    a = new int[2];
    a[0] = 50; a[1] = 60;
    return a;
}

public static void main(String args[]) {
    int[]a = {10,20,30};
    a = fun(a);
    System.out.println(a[0]);
}
```

o/p :- 50 :-



8:16 :-

Break till 8:20

Sorting : Arrangement of data basis some parameter.

Ex:- 1. $\{2, 3, 9, 12, 17, 19\}$

Parameter:- Magnitude ↑.

2. $\{19, 6, 5, 2, -1, -19\}$

Parameter:- Magnitud ↓.

Q.7

0 1 2 3 4

arr :- $\{1, 13, 9, 6, 12\}$ ✓

Count of factors :- 1, 2, 3, 4, 5

Parameter:- Count of factors ↑.

Note:- In today's class, we will be using in-built libraries to sort data.

Java :-

Arrays.sort(arr) # Static arrays

Collections.sort(arr) # ArrayList

C++ :- `sort(arr, arr + N);` # Static arrays

`sort(arr.begin(), arr.end());` # Vectors

Python :- `arr.sort();`

T.C → O(N log N)

S.C → O(N)

Problem 1

Given N elements, at every step remove an array element.

Cost to remove an element = Sum of array of elements present in an array

Find minimum cost to remove all elements.

ex:- $\{2, 1, 4\}$ (0, 1, 2)

Ways to removal

a)
1	1+2+4 = 7
2	2+4 = 6
4	4
	17

b)
2	1+2+4 = 7
4	1+4 = 5
1	1
	13

c)
4	4+2+1 = 7
2	1+2 = 3
1	1
	11

ans:

+ 3 more ;

1, 4, 2 → 15
2, 1, 4 → 16
4, 1, 2 → 12

Generalisation:-

ex:- $\{a, b, c, d\}$ (0, 1, 2, 3)

Let's remove in the order

	Cost
a	$a + b + c + d$
b	$b + c + d$
c	$c + d$
d	d

$a + 2b + 3c + 4d.$

↓

Can only be minimum

$a > b > c > d.$

Solution Approach

Sort the array in descending order.

Iterate on that array - I

add $(i+1) * A[i]$ to the ans;

Code

```
int findMinimumCost(int[] arr, int N) {
    reverse_sort(arr);          // N log N.
    int ans = 0;
    for (i = 0; i < N; i++) {
        ans += (i+1) * arr[i];  // N.
    }
    return ans;
}
```

T.C → O(N log N)
S.C → O(N)

Inbuilt Sorting also.

Problem 2

(distinct elements)

Given N array elements, calculate number of noble integers.

An element element in arr [ ] is said to be noble if { count of smaller elements = element itself }

ex:- { 1, -5, 3, 5, -10, 4 }

0 1 2 3 4 5

ans = 3

Q-8

{ -3, 0, 2, 5 }

0 1 2 3

ans = 1

Idea 1:- For every element A[i], iterate on the array to count smaller elements. If count(smaller) == A[i], increment ans.

T.C → $O(N^2)$.

Idea 2:-

ex:- { 1, -5, 3, 5, -10, 4 }
0 1 2 3 4 5

Sorted arr :- { -10, -5, 1, 3, 4, 5 }.

1. Sort the array.
2. Iterate on that array

If (A[i] == i) { ans++; }.

Code

```
1.  Sort(arr);

int ans = 0;
for (i=0 ; i < N; i++) {
    if (i == A[i]) {
        ans++;
    }
}
return ans;
```

T.C → O(NlogN)
S.C → O(N)
↓
Sorting Algo

Variation

What if the elements are not distinct ?

Q:- 9

arr :- {-10, 1, 1, 3, 100}

0 1 2 3 4

ans = 3 :.

arr :- {-10, 1, 1, 2, 4, 4, 4, 8, 10}

0 1 2 3 4 5 6 7 8

ans = 5 :.

arr :- {-3, 0, 2, 2, 5, 5, 5, 5, 8, 8, 10, 10, 10, 14}

0 1 2 3 4 5 6 7 8 9 10 11 12 13

ans = 7 :.

Observation

1. If any one of the duplicate elements is nobel, all other elements will also be nobel.

2. The first element of the duplicates, if (A[i] == i) then all the duplicates would also be nobel.

Approach:-

1. Sort the array in ascending order.

2. Iterate on the array.

a) A[i] == A[i-1]

↳ Repeated occurence of some duplicate element.

The A[i-1] will determine whether A[i] is noble or not:

b) A[i] ≠ A[i-1].

↳ First occurence of any duplicate sequence.

Index i will determine whether A[i] is noble or not:

In Assignments:

Noble Integer is defined as

Count of greater elements = Element itself.

Refer the code below:

Also, instead of counting, you have to validate.

Java

```java
public class Solution {
    public int solve(ArrayList<Integer> A) {

        // Sort in descending order
        Collections.sort(A, Collections.reverseOrder());

        int n = A.size();

        for (int i = 0; i < n; i++) {

            // Skip duplicates (only process first occurrence in descending order)
            if (i > 0 && A.get(i).equals(A.get(i - 1))) {
                continue;
            }

            // In descending order:
            // elements before index i = i (all are greater)
            if (A.get(i) == i) {
                return 1;
            }
        }

        return -1;
    }
}
```